UNIVERSIDADE DE MOGI DAS CRUZES

BACHARELADO EM ENGENHARIA DE SOFTWARE

HIGOR DE CASTRO VENANCIO DA SILVA 11252400484

JOÃO VITOR RODRIGUES 11261405984

Relatório Técnico StudyFlow

Mogi das Cruzes

2026

HIGOR DE CASTRO VENANCIO DA SILVA 11252400484

JOÃO VITOR RODRIGUES 11261405984

Relatório Técnico StudyFlow

Trabalho apresentado para o Prof. Fabiano Bezerra Menegidio sobre atividade relacionada à disciplina de Políticas de Segurança da Informação do curso de Bacharelado em Engenharia de Software

Mogi das Cruzes

2026

SUMÁRIO

SUMÁRIO

1. INTRODUÇÃO 3

2. DESCRIÇÃO DAS FERRAMENTAS ADOTADAS 4
2.1 Varredura de segredos: TruffleHog 4
2.2 SCA: Trivy 4
2.3 SAST: GitHub CodeQL 5
2.4 Política de bloqueio 5
2.5 Esteira do front-end 5

3. INTEGRAÇÃO COM O "REQUISITOS 1.PDF" 6

4. EVIDÊNCIAS DE FUNCIONAMENTO 7
4.1 Execução da esteira (Job Green) 7
4.2 Relatório do Trivy como artefato 8
4.3 Bloqueio por falha real e mitigação 9
4.4 Bloqueio do merge na branch principal 10
4.5 Evidências do Front-end 11

5. JUSTIFICATIVAS 12
5.1 TruffleHog 13
5.2 SCA: Trivy 13
5.3 SAST: GitHub CodeQL 13
5.4 SAST no front-end: Semgrep 13

6. CONCLUSÃO 13

1. INTRODUÇÃO

Em um ciclo de desenvolvimento tradicional, a segurança costuma ser verificada de forma precoce, seja em auditorias pontuais ou testes manuais. Isso permite que falhas como credenciais expostas, dependências vulneráveis e padrões inseguros de código cheguem à branch principal. A abordagem DevSecOps move essas verificações para dentro do fluxo de integração contínua, de modo que cada push e cada pull request seja validado automaticamente, sem depender de alguém lembrar de executar a análise.

No StudyFlow, essa automação foi implementada com GitHub Actions, cobrindo três frentes: varredura de segredos, análise de dependências (SCA) e análise estática do código (SAST). Qualquer achado de severidade alta ou crítica faz o job falhar e, combinado à proteção de branch, impede o merge da alteração na main.

2. DESCRIÇÃO DAS FERRAMENTAS ADOTADAS

2.1 Varredura de segredos: TruffleHog

Procura credenciais expostas no código e no histórico do Git, como chaves de API, tokens e senhas.

- Em push e pull request, compara apenas o intervalo de commits alterado (base e head), o que torna a verificação rápida.
- Semanalment e manualmente, varre o histórico completo do repositório, detectando segredos introduzidos em commits antigos.
- Usa --results=verified,unknown, reportando segredos confirmados como válidos ou que não puderam ser descartados, o que reduz falsos positivos.

2.2 SCA: Trivy

O Trivy analisa o `pom.xml` e cruza cada dependência com bases públicas de CVEs, filtrando severidades `HIGH` e `CRITICAL` com `--exit-code 1`.

Cada execução gera três saídas:

- relatório em texto (`trivy-report.txt`), publicado como artefato;

- relatório SARIF, enviado ao painel *Security* do GitHub;
- falha do job, quando há achado bloqueante.

2.3 SAST: GitHub CodeQL

O CodeQL analisa o código-fonte Java com o conjunto de regras security-extended, que cobre categorias como injeção de SQL, falhas de controle de acesso e uso inseguro de criptografia. A análise exige compilar o projeto (mvn package, JDK 25, mesma versão do pom.xml).

Como o CodeQL não bloqueia o build por padrão, foi adicionado um passo próprio que lê o SARIF com jq e falha o job quando existe achado com CVSS ≥ 7.0 ou, na ausência de nota CVSS, classificado como error. Se nenhum SARIF for gerado, o job também falha, evitando que um erro na análise passe como "código limpo".

2.4 Política de bloqueio

Frente	Ferramenta	Critério de falha
Segredos	TruffleHog	Segredo verificado ou não descartável
SCA	Trivy	Vulnerabilidade HIGH ou CRITICAL (`exit-code 1`)
SAST	CodeQL + passo `jq`	CVSS ≥ 7.0, ou nível `error` sem CVSS

O token do workflow segue o princípio do menor privilégio (`contents: read` por padrão), e a publicação no painel Security é desabilitada em PRs vindos de forks, que não têm essa permissão.

2.5 Esteira do front-end

- O repositório do front-end (StudyFlow-FrontEnd) possui uma esteira própria, definida em .github/workflows/devsecops-pipeline.yml, executada em push e pull request para a main. Ela segue as mesmas três frentes de segurança do back-end:

  Segredos (TruffleHog): varre todo o histórico do Git e falha se encontrar qualquer segredo.

- Dependências (Trivy): analisa o package-lock.json e falha ao encontrar vulnerabilidade HIGH ou CRITICAL.
- Código (Semgrep): analisa o código JavaScript/React e falha ao encontrar achado de severidade ERROR. O CodeQL roda como análise complementar, com os alertas no painel Security.

Um job final, o Security Gate, só é aprovado quando os três jobs bloqueantes passam. É esse check que a regra de proteção da main exige para liberar o merge.

3. Integração com o "Requisitos 1.pdf"

O documento de requisitos define controles que, uma vez implementados, podem regredir a qualquer alteração futura. A esteira transforma parte desses requisitos em verificações contínuas:

Requisito	Como a esteira contribui
3.6 Chaves criptográficas protegidas / **6.4** Gestão de credenciais	O TruffleHog detecta qualquer commit que exponha o segredo do JWT ou a chave da API de email, que hoje ficam apenas em variáveis de ambiente
6.7 Ameaças e vulnerabilidades / **3.5** Algoritmos adequados	O Trivy monitora JJWT, Spring Security e o driver PostgreSQL. O achado real desta atividade (bypass de proteção MITM no SCRAM-SHA-256-PLUS do driver) afeta a segurança da conexão com o banco
1.x Autenticação e sessão / **5.x** Auditoria	O CodeQL varre os fluxos de login, JWT, logs e consultas ao banco em busca de injeção, controle de acesso falho e criptografia fraca
6.9 / 6.10 Testes de segurança	Cada execução gera evidência rastreável: relatório do Trivy, SARIF e histórico de runs

realizados e documentados	
4.x LGPD	A prevenção de vazamento de credenciais e a correção de vulnerabilidades reduzem o risco de acesso indevido a dados pessoais

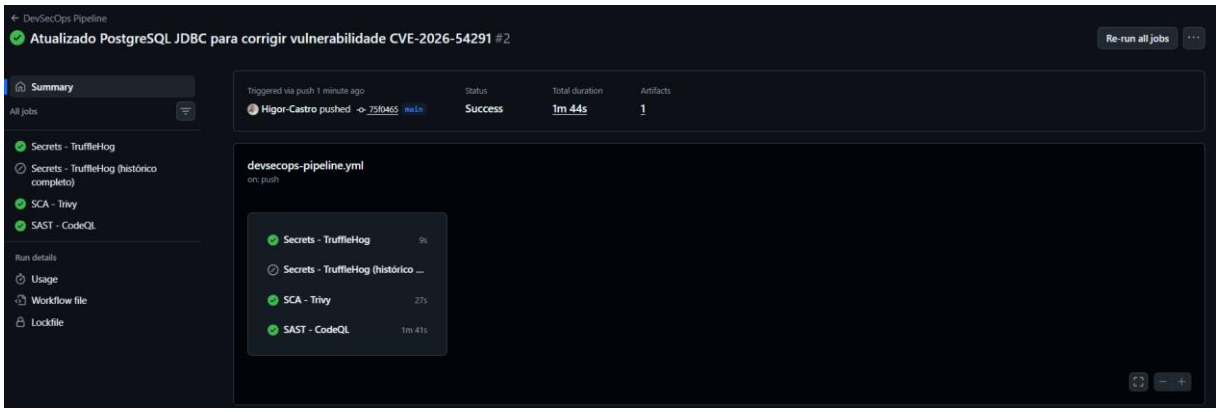
Na prática, a esteira cumpre três papéis: **auditoria** (todo run fica registrado), **conformidade** (os controles não regridem silenciosamente) e **proteção contínua** (o merge só ocorre com os três checks verdes).

Contribuição do front-end:

Na prática, a esteira cumpre três papéis: auditoria (todo run fica registrado), conformidade (os controles não regridem silenciosamente) e proteção contínu**a** (o merge só ocorre com os três checks verdes).

4. EVIDÊNCIAS DE FUNCIONAMENTO

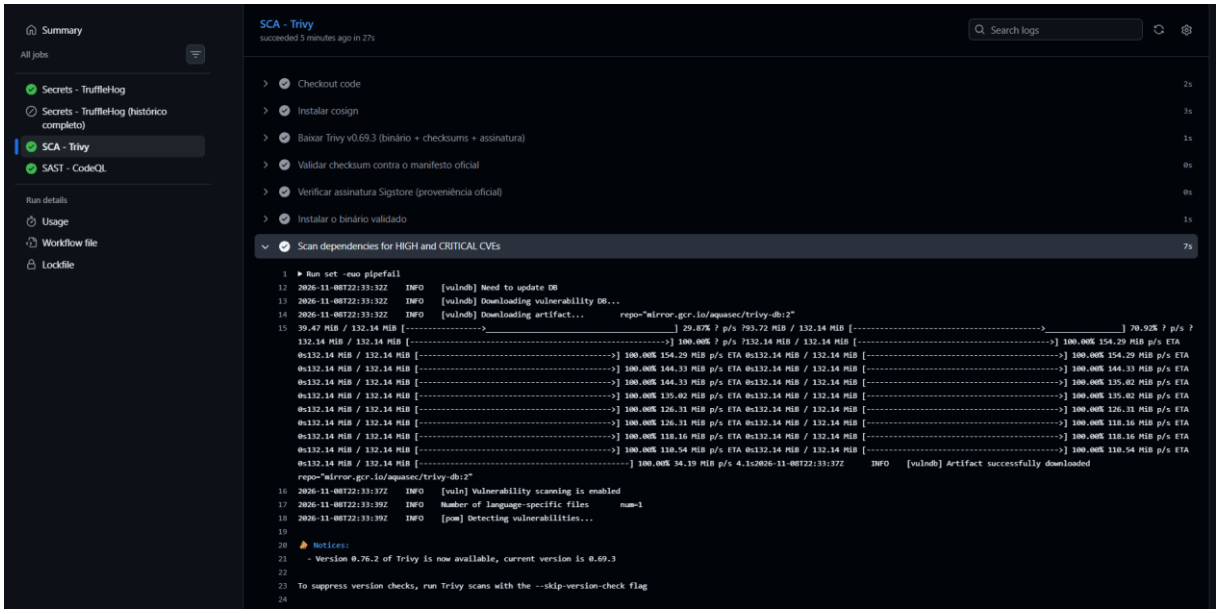
4.1 Execução da esteira (Job Green)



Execução da esteira demonstra que os três Jobs de segurança foram executados com sucesso.

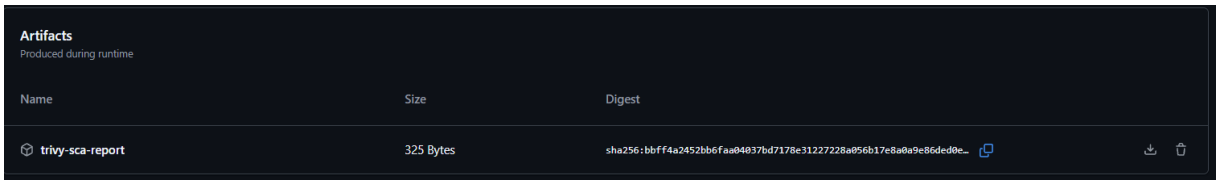


Validação do CodeQL demonstra que não foram encontrados achados bloqueantes na análise estática do código.


Summary
All jobs
Secrets - TruffleHog
Secrets - TruffleHog (histórico completo)
SCA - Trivy
SAST - CodeQL
Run details
Usage
Workflow file
Lockfile
SCA - Trivy
succeeded 5 minutes ago in 27s
Search logs
Checkout code 2s
Instalar cosign 3s
Baixar Trivy v0.69.3 (binário + checksums + assinatura) 1s
Validar checksum contra o manifesto oficial 0s
Verificar assinatura Sigstore (proveniência oficial) 0s
Instalar o binário validado 1s
Scan dependencies for HIGH and CRITICAL CVEs 7s
1 ▶ Run set -euo pipefail
12 2026-11-08T22:33:32Z INFO [vulndb] Need to update DB
13 2026-11-08T22:33:32Z INFO [vulndb] Downloading vulnerability DB...
14 2026-11-08T22:33:32Z INFO [vulndb] Downloading artifact... repo="mirror.gcr.io/aquasec/trivy-db:2"
15 39.47 MiB / 132.14 MiB [----------->_____] 29.87% ? p/s ?93.72 MiB / 132.14 MiB [----------->_____] 70.92% ? p/s ?
132.14 MiB / 132.14 MiB [-----------------------

4.2 Relatório do Trivy como artefato

O relatório é publicado como artefato baixável (trivy-sca-report), permitindo revisão sem acessar o painel Security.



Artefato Trivy demonstra o armazenamento do relatório de vulnerabilidades para consulta posterior.

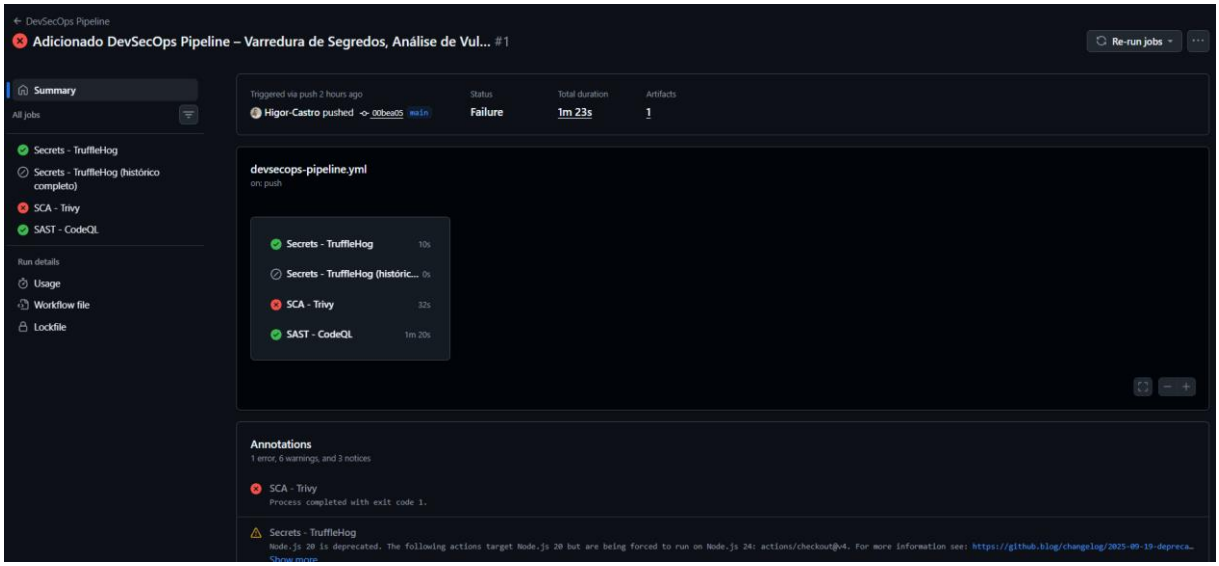
4.3 Bloqueio por falha real e mitigação

Na primeira execução, o Trivy identificou 1 vulnerabilidade HIGH e a esteira falhou conforme projetado:

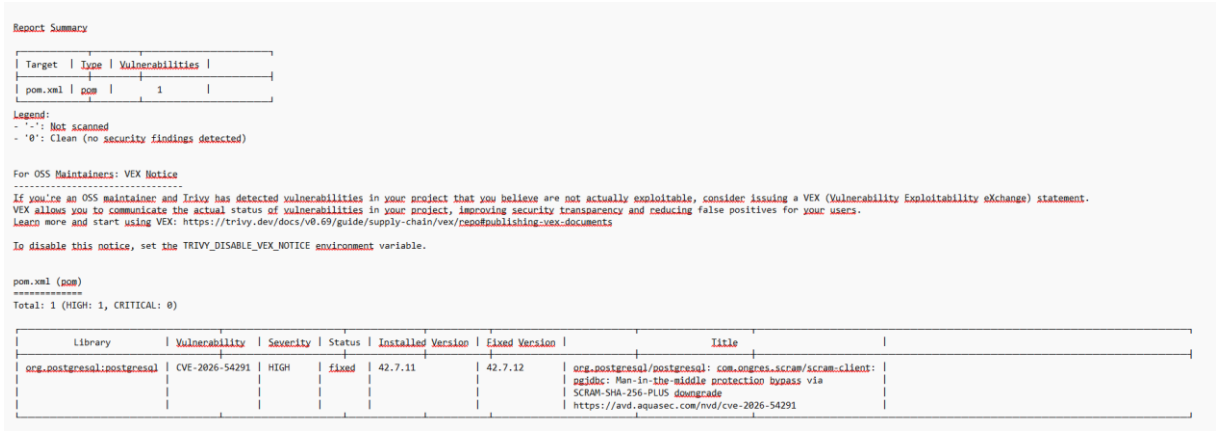
Biblioteca	CVE	Severidade	Versão instalada	Versão corrigida
org.postgresql:postgresql	CVE-2026-54291	HIGH	42.7.11	42.7.12

A falha permite contornar a proteção contra interceptação (man-in-the-Middle) afetando a conexão entre aplicação e banco.

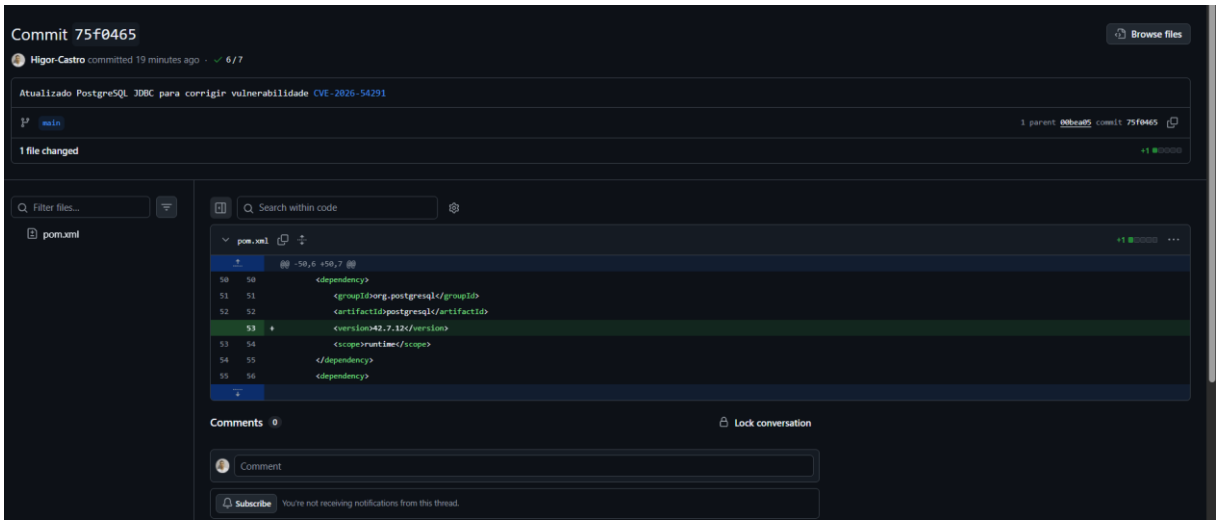
Mitigação: como a versão do driver é gerenciada pelo Spring Boot, foi definida a propriedade <version>42.7.12</version> no pom.xml. Após o commit, o Trivy executou sem achados e a esteira ficou verde.



Falha na esteira demonstra que a pipeline bloqueou a execução após identificar uma vulnerabilidade de alta severidade.



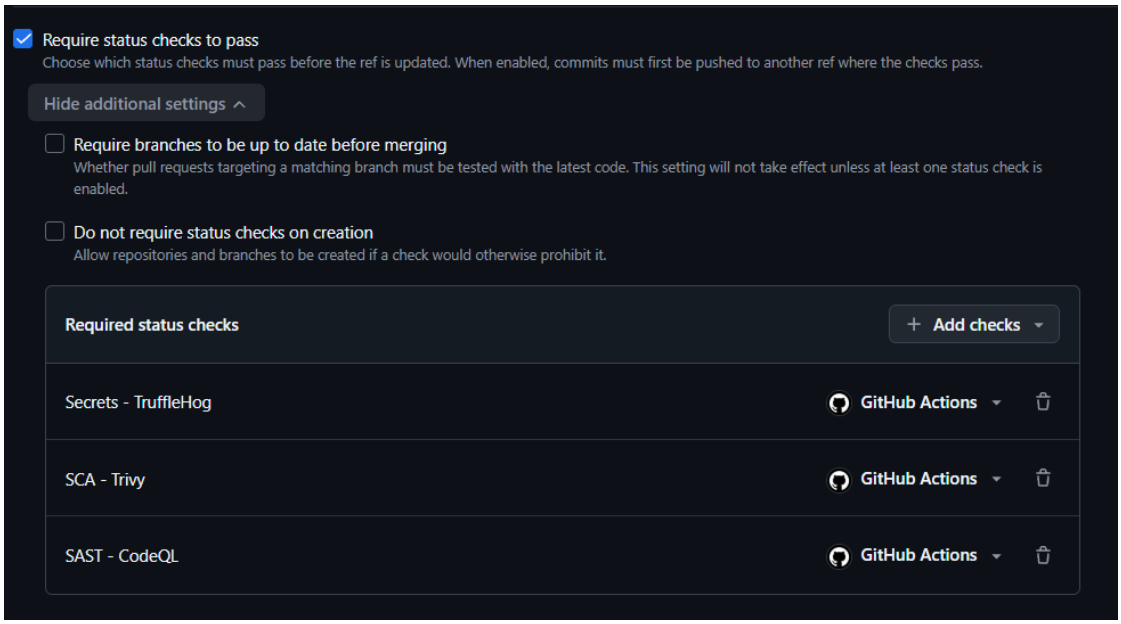
Vulnerabilidade identificada apresenta a CVE-2026-54291 detectada pelo Trivy na dependência do PostgreSQL.



Correção da dependência demonstra a atualização da versão do PostgreSQL para corrigir a vulnerabilidade identificada.

4.4 Bloqueio do merge na branch principal

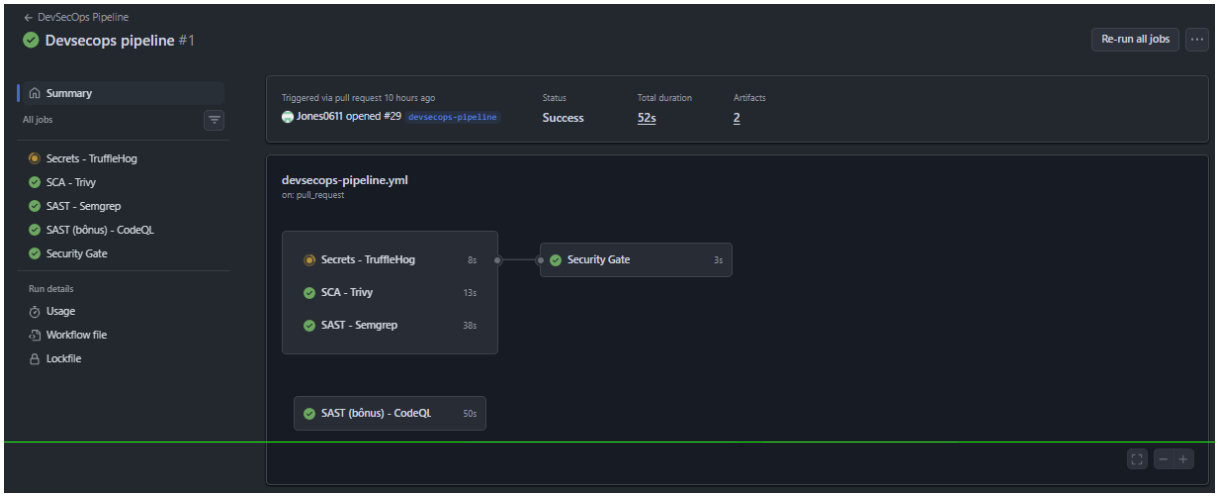
Para que o check vermelho impeça o merge, a branch `main` exige que os três jobs passem.



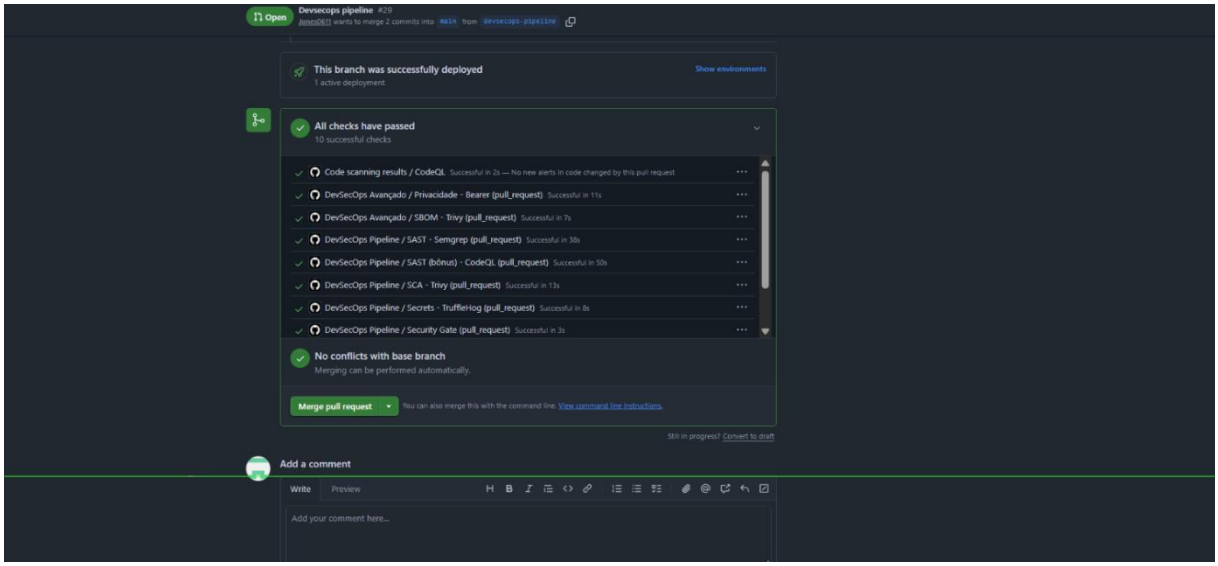
Proteção da branch Demonstra a configuração de regras que exigem a aprovação dos checks antes do merge na Branch principal.

4.5 Evidências do Front-end

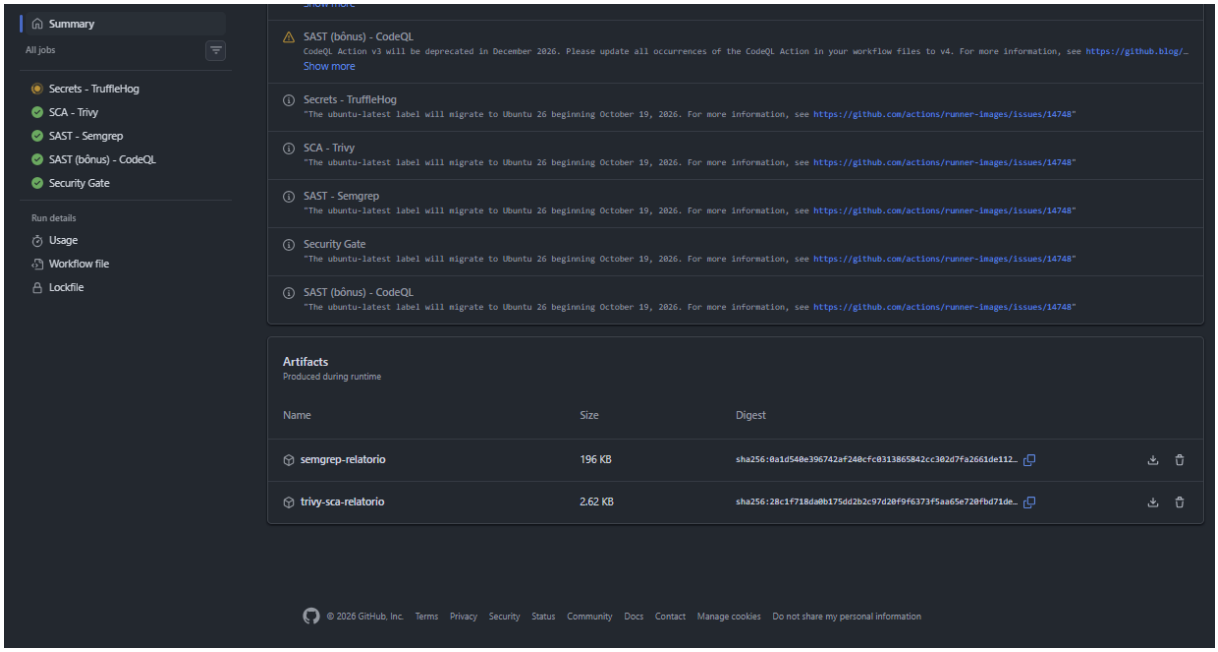
Após o merge do pull request que adicionou a esteira, o workflow DevSecOps Pipeline foi executado na main com todos os jobs concluídos com sucesso.



Execução da esteira do front-end demonstra que os jobs de segurança foram executados com sucesso.



Checks do pull request que adicionou a esteira, todos aprovados.



Cada execução guarda os relatórios das ferramentas como artefatos baixáveis. O relatório do Trivy (trivy-sca-relatorio) não registra nenhuma vulnerabilidade HIGH ou CRITICAL com correção disponível no package-lock.json. O relatório do Semgrep (semgrep-relatorio) registra 16 observações de baixa gravidade, nenhuma de severidade ERROR, portanto não bloqueantes: 14 referentes a actions referenciadas por tag mutável (como actions/checkout@v4) e 2 referentes ao dependabot.yml sem período de espera (cooldown).

5. Justificativas

5.1 TruffleHog

O StudyFlow depende de segredos sensíveis (chave de assinatura do JWT, chave da API de email), e vazá-los comprometeria a autenticação. O TruffleHog é gratuito, open-source e verifica se o segredo encontrado está ativo. No front-end, o scan falha com qualquer segredo encontrado, verificado ou não, já que a interface não deve carregar nenhuma credencial.

5.2 SCA: Trivy

Além do requisito de bloqueio, a instalação do Trivy foi feita com verificação de integridade e origem: checksum SHA-256 contra o manifesto oficial e assinatura Sigstore validada com Cosign. Isso evita que a própria ferramenta de segurança vire um ponto de entrada de código malicioso (risco de cadeia de suprimentos). A versão também é fixada, garantindo comportamento reprodutível. No front-end, o Trivy também é instalado em versão fixa, com verificação de checksum SHA-256.

5.3 SAST: GitHub CodeQL

O repositório é público, o que habilita o CodeQL gratuitamente com publicação dos alertas na aba Security. Para SCA foi escolhido o Trivy (em vez do Dependabot) por permitir fixar o critério de bloqueio e gerar artefatos baixáveis.

5.4 SAST no front-end: Semgrep

No front-end foi usado o Semgrep como SAST bloqueante porque é open-source, roda direto no runner e informa a severidade de cada regra, o que permite falhar o job com --severity ERROR --error, como pede o enunciado. As regras de JavaScript, React e OWASP Top 10 cobrem o código do front-end sem precisar compilar o projeto. O CodeQL foi mantido como análise complementar, pois não bloqueia o build por padrão e serve melhor como evidência no painel Security.

6. Conclusão

A implementação da pipeline DevSecOps no StudyFlow permitiu automatizar verificações de segurança durante o desenvolvimento, utilizando TruffleHog, Trivy e CodeQL. Os testes demonstraram a identificação de vulnerabilidades, a validação das correções e o bloqueio de alterações que não atendem aos critérios de segurança. Dessa forma, a solução contribui para tornar o desenvolvimento mais seguro, contínuo

e confiável.

No front-end, a esteira identificou três vulnerabilidades de severidade alta em dependências npm, e a alteração só foi aceita depois da correção, com o Security Gate verde.